

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS — PUC-CAMPINAS
PADRÕES E ARQUITETURA DE SOFTWARE

Controle de Gastos Pessoais

Projeto de software com arquitetura justificada,
SOLID, Clean Code e padrões de projeto GoF

Integrantes do grupo:

Gabriel Lopes Londe Rodrigues — RA: 24005520

Nicolas Marques Linares — RA: 24015266

Matheus Roçafa — RA: 23028589

Felipe Andretta — RA: 23007744

João Victor Lunardini — RA: 23028241

Professor: Douglas Henrique Siqueira Abreu

Repositório: https://github.com/Lopesloro/Gastos_Pessoais

Data: Junho de 2026

Sumário

1. Introdução
2. Atributos de Qualidade e Decisões Arquiteturais (ADRs)
3. Estilo Arquitetural (plano macro e interno)
4. Aplicação dos Princípios SOLID
5. Clean Code
6. Padrões de Projeto GoF
7. Design de API
8. Diagramas e Modelos
9. Conclusões
10. Referências Bibliográficas

1. Introdução

1.1 Apresentação e justificativa

O **Controle de Gastos Pessoais** é uma aplicação que permite ao usuário registrar suas **receitas** e **despesas** organizadas por **categoria**, acompanhar o **resumo mensal** (total de receitas, despesas e saldo) e receber **alertas** quando os gastos do mês ultrapassam um limite de orçamento. O projeto foi escolhido como *projeto novo*, com escopo deliberadamente delimitado para permitir a aplicação **clara e demonstrável** dos conceitos da disciplina — arquitetura, atributos de qualidade, SOLID, Clean Code e padrões GoF — sem que a complexidade acidental do domínio ofusque as decisões de design.

1.2 Objetivos

Objetivo geral: desenvolver um software funcional cujas decisões arquiteturais sejam conscientes, justificadas e rastreáveis na documentação e no código.

Objetivos específicos:

- Registrar receitas e despesas por categoria, com validação de dados.
- Calcular resumos (mensal e por categoria) de forma extensível.
- Alertar o usuário ao estourar o orçamento mensal.
- Expor uma API REST documentada via OpenAPI.
- Evidenciar, com código real, SOLID, Clean Code e três padrões GoF.

1.3 Problema e público-alvo

Problema: pessoas físicas frequentemente perdem o controle de para onde vai seu dinheiro e descobrem tarde que estouraram o orçamento. O sistema ataca esse problema centralizando os lançamentos, resumindo-os por período/categoria e **avisando proativamente** sobre o estouro do limite.

Público-alvo: usuário individual que deseja uma ferramenta simples de finanças pessoais, sem a complexidade de softwares contábeis.

2. Atributos de Qualidade e Decisões Arquiteturais

2A. Atributos de qualidade prioritários (ISO/IEC 25010:2023)

Dos oito atributos do modelo de qualidade de produto da norma, três foram eleitos prioritários, com justificativa, decisão arquitetural associada e métrica observável.

1º — Manutenibilidade

Por que, e não outro: o objetivo do projeto é *evoluir e demonstrar design* (novos relatórios, novos tipos de transação, troca de armazenamento). Performance e escalabilidade não são prioridade num cenário mono-usuário local.

Decisões: Clean Architecture em camadas (ADR-001), padrões GoF isolando pontos de variação, SOLID (sobretudo Inversão de Dependência).

Métrica: adicionar um novo relatório ou tipo de transação deve exigir alteração em ≤ 2 arquivos e **nenhuma** modificação em classes existentes (apenas adição).

2º — Testabilidade

Por que: sustentar a evolução com segurança exige provar as regras sem depender de banco/servidor; também é valorizada nos critérios de avaliação.

Decisões: serviços recebem repositórios pela abstração; domínio sem dependências de framework.

Métrica: regras centrais cobertas por testes que rodem sem infraestrutura em < 5 s. Atual: **9 testes, ~1,7 s.**

3º — Confiabilidade (alertas de orçamento)

Por que: a função de valor é evitar o estouro do orçamento; não avisar seria uma falha funcional silenciosa.

Decisões: padrão Observer (ADR-004); validação de invariantes no domínio com erro explícito (HTTP 422).

Métrica: taxa de alertas perdidos = **0%** (toda despesa que excede o limite mensal dispara alerta) — verificado por teste e pelo campo `alerta_orcamento` da API.

2B. Registro de Decisões Arquiteturais (ADRs)

Cinco ADRs no formato canônico estão versionados na pasta `/adrs` do repositório. Resumo:

ADR	Título	Status	Essência
001	Clean Architecture em camadas	Accepted	Regra de dependência para dentro; portas no domínio.
002	Factory Method para transações	Accepted	Centralizar criação de Receita/Despesa.
003	Strategy para resumos	Accepted	Algoritmos de resumo intercambiáveis.
004	Observer para alertas	Accepted	Desacoplar a regra do canal de alerta.
005	SQLite implementado; PostgreSQL futuro	Accepted (revisado)	SQLite integrado em <code>infrastructure/repositorios_sqlite.py</code> ; dados persistem entre execuções. ADR registra 3 fases da decisão.

O **ADR-005** atende à recomendação do enunciado de registrar pelo menos uma **decisão revertida/modificada** ao longo do desenvolvimento.

3. Estilo Arquitetural

3A. Plano macro — decomposição em unidades implantáveis

O sistema é um **monólito modular** (uma única unidade implantável, internamente organizada em módulos/camadas), **não** microserviços.

Justificativa (ligada à seção 2A):

- O atributo prioritário é **manutenibilidade**, não escalabilidade — e escalar partes de forma independente é a principal razão para microserviços.
- **Equipe pequena** e domínio enxuto: microserviços trariam custo operacional (deploys múltiplos, rede, observabilidade distribuída) sem benefício para os atributos declarados.
- O monólito **modular** preserva a evolução: fronteiras claras permitem extrair um serviço no futuro, se necessário.

3B. Plano interno — organização do código

Adotou-se **Clean Architecture** (Martin) em quatro camadas, com a regra de dependência apontando sempre para dentro. As interfaces de repositório (portas) ficam no domínio; a infraestrutura as implementa (adaptadores) — variante hexagonal de portas e adaptadores.

Camada	Pasta	Responsabilidade
Domínio	domain/	Entidades, regras invariantes e portas (interfaces de repositório).
Aplicação	application/	Casos de uso que orquestram entidades e padrões.
Padrões	patterns/	Factory, Strategy, Observer.
Infraestrutura	infrastructure/	Adaptadores concretos: <code>repositorios_sqlite.py</code> (produção, persiste em <code>data/gastos.db</code>) e <code>repositorios_memoria.py</code> (testes).
Interface	api/	FastAPI: rotas, DTOs e composition root (injeção de dependência).

Diagrama de componentes (estilo C4) na Seção 8.

4. Aplicação dos Princípios SOLID

S — Single Responsibility

Princípio: uma classe deve ter um único motivo para mudar. **Aplicação:** `ResumoMensualStrategy` só calcula o resumo mensal; `TransacaoService` só orquestra o registro; os DTOs (`api/schemas.py`) separam a representação HTTP das entidades. **Efeito:** mudar o JSON da API não toca a regra de negócio.

O — Open/Closed

Princípio: aberto para extensão, fechado para modificação. **Aplicação:** novos relatórios entram como novas estratégias; o serviço não muda.

```
class ResumoService:
    def gerar(self, strategy: ResumoStrategy) → dict:
        return strategy.calcular(self._transacao_repo.listar())
```

Efeito: adicionar "resumo anual" não altera código testado.

L — Liskov Substitution

Princípio: subtipos substituíveis pelo tipo base. **Aplicação:** `Receita` e `Despesa` são `Transacao`; diferem apenas no sinal de `valor_no_fluxo`.

```
Receita(...).valor_no_fluxo    # +valor
Despesa(...).valor_no_fluxo    # -valor
```

Efeito: `ResumoPorCategoriaStrategy` soma `valor_no_fluxo` sem checar o tipo concreto.

I — Interface Segregation

Princípio: clientes não dependem de métodos que não usam. **Aplicação:** `CategoriaRepository` e `TransacaoRepository` são interfaces separadas; `AlertaObserver` expõe um único método.

Efeito: cada cliente depende só do que precisa.

D — Dependency Inversion

Princípio: alto nível depende de abstrações, não de detalhes. **Aplicação:**

```
# application/servicos.py (alto nível)
def __init__(self, transacao_repo: TransacaoRepository): ...
```

```
# infrastructure/repositorios_sqlite.py (baixo nível)
class TransacaoRepositorySQLite(TransacaoRepository): ...

# api/dependencias.py (composition root injeta a concreta)
TransacaoService(_transacao_repo, _categoria_repo, _monitor)
```

Efeito: torna o ADR-005 (troca de banco) barato e os testes independentes de infraestrutura.

5. Clean Code

- **Nomes que revelam intenção:** `valor_no_fluxo` (valor com sinal), `MonitorDeOrçamento.avaliar()`, `AlertaObserver.notificar()`.
- **Funções pequenas, um nível de abstração:** `TransacaoService.registrar()` lê como narrativa; detalhes em `_total_despesas_do_mes()`.
- **Sem números/textos mágicos:** limite de orçamento em constante nomeada `LIMITE_MENSAL`; tipos em `Enum TipoTransacao`.
- **Sem duplicação (DRY):** conversão entidade→DTO centralizada em `_to_transacao_out()`; lógica de qual subclasse criar só na `TransacaoFactory`.
- **Baixo acoplamento, alta coesão:** camadas isoladas; domínio sem `import` de FastAPI.
- **Ausência de comentários redundantes:** o código se autoexplica por nomes descritivos e funções pequenas; não há comentários repetindo o que o código já diz (princípio de R. C. Martin — "o comentário é, no melhor caso, um mal necessário").
- **Tratamento de erros explícito:** invariantes validadas em `__post_init__` (falha cedo); API traduz `ValueError` em HTTP 422.
- **Testabilidade:** 9 testes automatizados cobrindo domínio, Factory, Observer, Strategy e API.

6. Padrões de Projeto GoF

6.1 Factory Method (Criacional)

Problema: a API recebe o tipo como texto; espalhar `if/else` para instanciar `Receita / Despesa` acoplaria os chamadores às subclasses. **Solução:** centralizar a criação na fábrica.

```
class TransacaoFactory:
    @staticmethod
    def criar(tipo, descricao, valor, categoria, data=None):
        construtores = {TipoTransacao.RECEITA: Receita,
                        TipoTransacao.DESPESA: Despesa}
        return construtores[tipo](descricao=descricao, valor=valor,
                                categoria=categoria, data=data or date.today())
```

Benefício: Aberto/Fechado (novo tipo = 1 linha). **Custo:** uma indireção a mais.

6.2 Strategy (Comportamental)

Problema: múltiplas formas de resumir (mensal, por categoria, futura anual) virariam um `if/elif` frágil. **Solução:** cada algoritmo é uma estratégia intercambiável.

```
class ResumoStrategy(ABC):
    @abstractmethod
    def calcular(self, transacoes: list[Transacao]) → dict: ...
```

Benefício: novo relatório sem tocar nos existentes. **Custo:** uma classe por relatório.

6.3 Observer (Comportamental)

Problema: alertar o estouro de orçamento sem acoplar a regra ao canal (log/e-mail/push).

Solução: `MonitorDeOrçamento` (Subject) notifica observadores inscritos.

```
class MonitorDeOrçamento:
    def avaliar(self, total_gasto_mes: Decimal) → bool:
        if total_gasto_mes > self.limite_mensal:
            for obs in self._observadores:
                obs.notificar(total_gasto_mes, self.limite_mensal)
            return True
        return False
```

Benefício: novo canal de alerta não muda o disparo. **Custo:** indireção entre quem dispara e quem trata.

Diagrama de classes dos três padrões na Seção 8.

7. Design de API

Estilo: REST com **OpenAPI 3.x** (gerado automaticamente pelo FastAPI; arquivo `openapi/swagger.yaml` no repositório; UI interativa em `/docs`). O domínio é orientado a recursos com CRUD claro — REST mapeia isso naturalmente; GraphQL/gRPC seriam sobredimensionados.

Método	Rota	Descrição
POST	<code>/api/v1/categorias</code>	Cria categoria
GET	<code>/api/v1/categorias</code>	Lista categorias
POST	<code>/api/v1/transacoes</code>	Registra receita/despesa (avalia orçamento)
GET	<code>/api/v1/transacoes</code>	Lista transações
GET	<code>/api/v1/resumo/mensal</code>	Resumo mensal (Strategy)
GET	<code>/api/v1/resumo/por-categoria</code>	Saldo por categoria (Strategy)

Versionamento: por caminho de URL (`/api/v1`); mudanças retrocompatíveis não incrementam a versão; quebras geram `/api/v2` com transição.

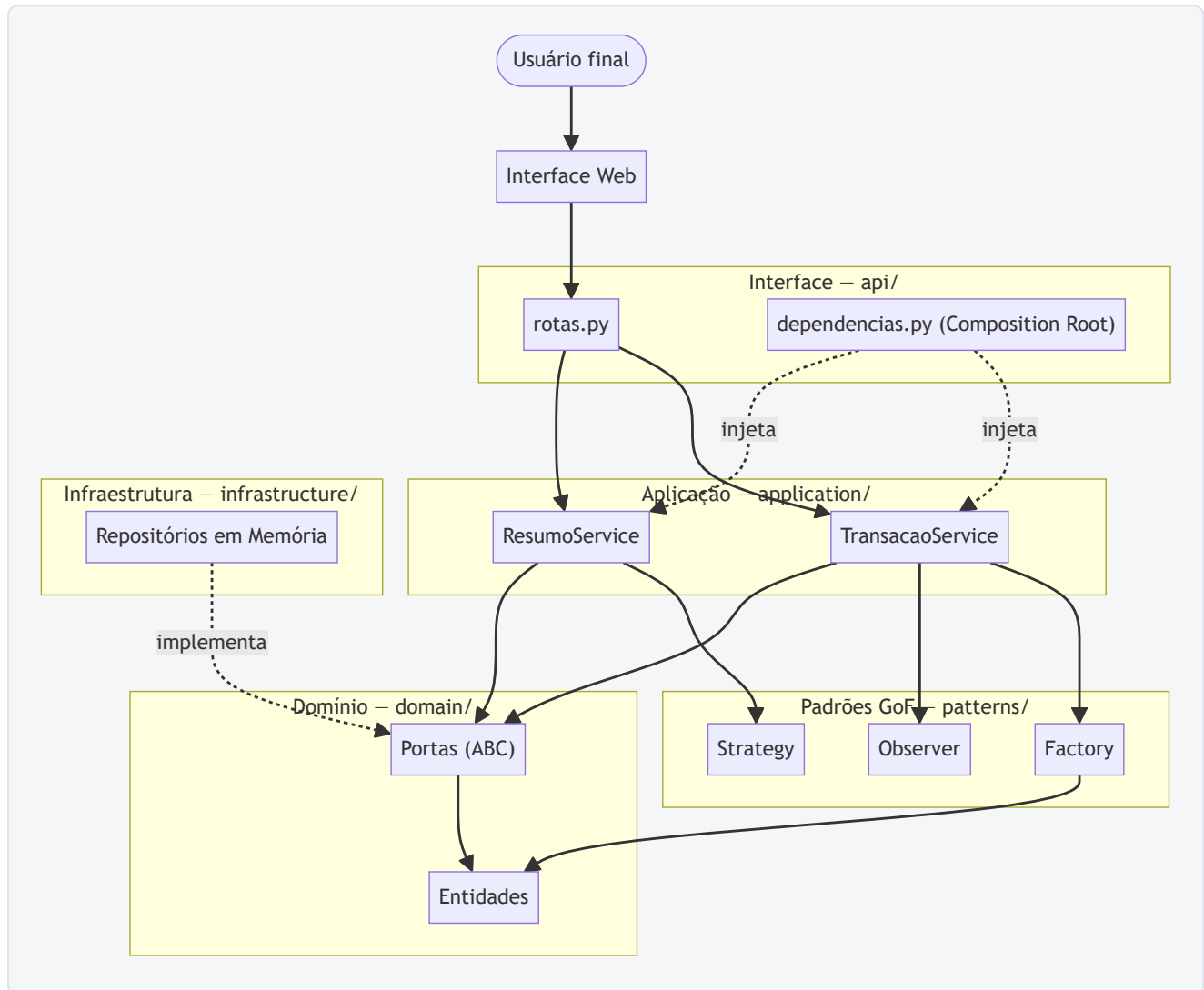
Erros: status HTTP + corpo `{"detail": " ... "}`; validação e violação de regra → `422`; rota inexistente → `404`; criação → `201`.

Paginação (planejada): `offset/limit` via query string, retrocompatível. **Autenticação:** ausente no escopo mono-usuário; evolução planejada com OAuth2 + JWT.

8. Diagramas e Modelos

Fontes versionadas em `/diagrams` (Mermaid). Reproduzidos abaixo.

8.1 Componentes (estilo C4)

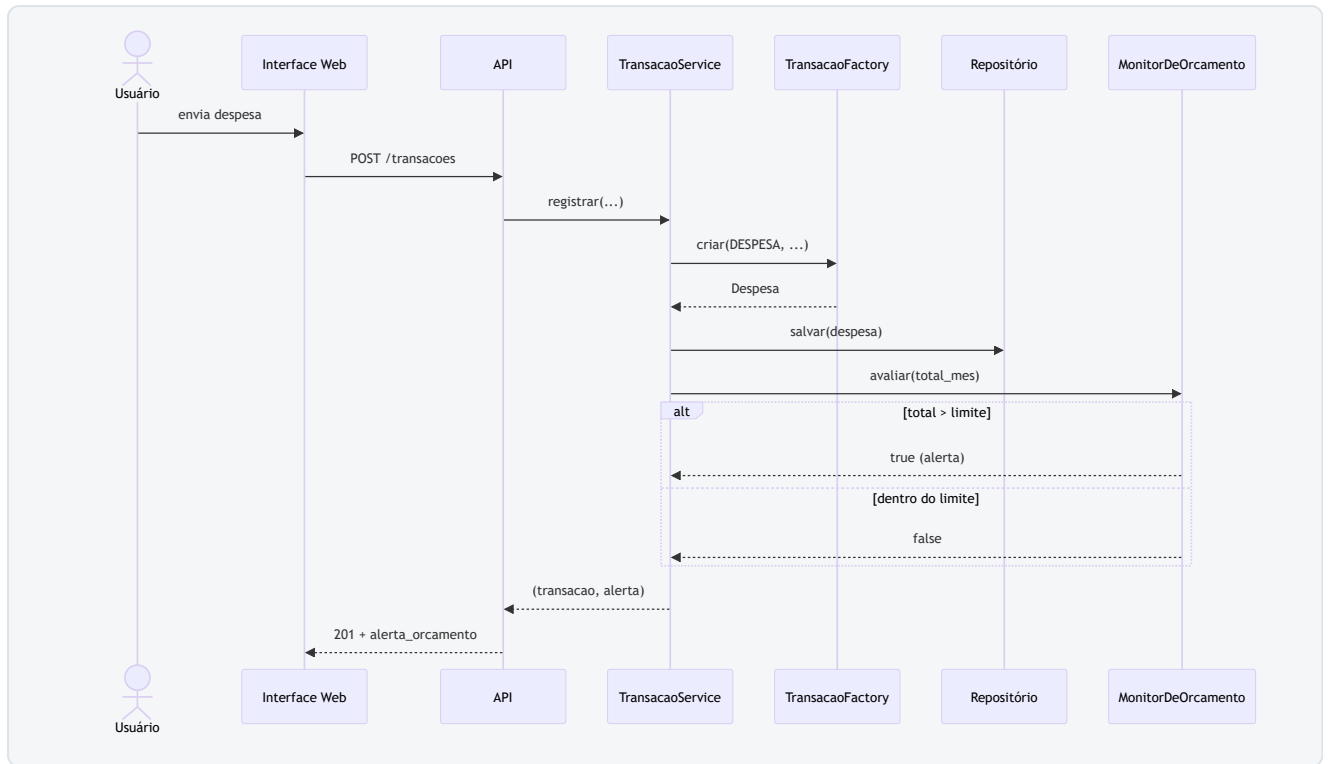


8.2 Classes — padrões GoF



Syntax error in text
mermaid version 10.9.6

8.3 Sequência — registrar uma despesa



9. Conclusões

Resultados. O sistema cumpre seus objetivos: registra receitas/despesas, gera resumos e alerta sobre estouro de orçamento, com uma API REST documentada e interface web. A arquitetura em camadas, somada a SOLID e aos três padrões GoF, resultou num código de baixo acoplamento e alta testabilidade — comprovado por 9 testes que rodam sem infraestrutura.

Efetividade das práticas. A Inversão de Dependência foi a decisão de maior retorno: tornou a reversão do ADR-005 trivial e os testes independentes de banco. O Strategy e o Observer pagaram seu custo de indireção ao tornar resumos e alertas extensíveis sem modificar código existente.

Limitações e próximos passos.

- **Concorrência:** o SQLite atende ao escopo mono-usuário, mas não suporta múltiplos escritores simultâneos; a evolução para PostgreSQL já está viabilizada pela porta abstrata (trocar uma classe).
- **Autenticação/multiusuário:** fora do escopo atual; evolução com OAuth2 + JWT.
- **Cobertura de testes:** ampliar para casos de borda e incluir medição de cobertura.
- **O que faríamos diferente:** introduzir a porta de persistência com o adaptador SQLite desde o primeiro commit, em vez de começar em memória e migrar depois — embora a migração tenha custado apenas uma classe nova, justamente por causa da Inversão de Dependência.

10. Referências Bibliográficas

ABREU, Douglas Henrique Siqueira. **Padrões e Arquitetura de Software**. [Material didático da disciplina]. Campinas: PUC-Campinas, 2026.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Design Patterns**: elements of reusable object-oriented software. Reading: Addison-Wesley, 1994.

MARTIN, Robert C. **Clean Code**: a handbook of agile software craftsmanship. Upper Saddle River: Prentice Hall, 2008.

MARTIN, Robert C. **Clean Architecture**: a craftsman's guide to software structure and design. Upper Saddle River: Prentice Hall, 2017.

FOWLER, Martin. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010:2023**: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Genebra: ISO, 2023.

NYGARD, Michael. **Documenting Architecture Decisions**. 2011. Disponível em: cognitect.com. Acesso em: jun. 2026.

ISO/IEC/IEEE. **ISO/IEC/IEEE 42010:2011**: Systems and software engineering — Architecture description. Genebra: ISO, 2011.